

Ghost of the Machine

- **Time limit:** 10 minutes
- **Baseline score:** 28.6
- **Environment:** one GPU (≈16 GB VRAM), no internet
- **Solution size:** `solution.ipynb` ≤ 20 MB
- **Storage:** 5 GB
- **Pretrained models:** only [bge-base-en-v1.5](#) — a text **encoder** (embedding model).

Task

Strange things are happening at the National Archive of Kazakhstan. Librarians say that some books used to end differently, but no one can prove it — every copy is the same, and every story still makes sense. You are invited as an AI researcher to locate the changes.



A passage begins as human-written text and, at some point, silently switches to a continuation generated by a language model. Read as a whole, it looks like one coherent piece — but somewhere in the middle the author changes from a person to a machine. Your task is to **find that switch: the character index where the human part ends and the machine part begins**.

Each sample is a single string `text`. There is exactly one boundary. Everything before it is human; everything from it onward is machine-generated.

Dataset

Plain-text English passages, one boundary each.

- **Part A** (before the boundary): an excerpt of human-written text.
- **Part B** (from the boundary on): a continuation produced by a language model, conditioned on Part A.
- Each side is at least 180 words; total length is ~500–800 words.
- The **boundary_char_index** is the index of the **first character of Part B**: `text[boundary_char_index:]` is exactly the machine part, and `text[:boundary_char_index]` is the human part together with the single space that separates the two.

What you get

You receive **two folders**:

Folder	Samples	answers.jsonl?	Use it to
dataset/train/	1,221	☐ included	train / fine-tune your method
dataset/test_public/	380	☐ included (dev copy)	run your pipeline and self-score locally

At **grade time** your `dataset/test_public/` folder is **replaced by a hidden evaluation set**. It has the same format but **without answers.jsonl**. Your notebook is re-run on it, and the `answers.jsonl` it produces is scored.

- The public leaderboard uses a hidden **test_leaderboard_a** set (380 samples).
- The final ranking uses a hidden **test_leaderboard_b** set (380 samples).

All three evaluation sets are the same size and drawn from the same distribution as `train`, so your local `dataset/test_public/` score is a reasonable estimation for your leaderboard score.

On-disk format

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- Ids in `answers.jsonl` match ids in `data.jsonl`.
- `dataset/train/` (with answers) is available whenever you train or fine-tune.

Output (submission format)

You submit **a single notebook, which must be named `solution.ipynb`**. This exact file name is required. Anything else is rejected without being run.

Your notebook must **read `dataset/test_public/data.jsonl`** and write a single file **`answers.jsonl`** at the repository root — one JSON object per line, mapping each sample id to your predicted boundary character index:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` must be an **integer in `[0, len(text)]`**.
- Every id in `dataset/test_public/data.jsonl` should appear exactly once. A sample missing from `answers.jsonl` (or with a non-integer / out-of-range value) scores 0 for that sample.

Scoring

For each sample, let p be your predicted index and t be the true boundary. The per-sample score decays exponentially with the character distance:

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

This leads to the following behavior of the score:

- **≈1.0** — exact boundary character;
- **≈0.78** — 25 characters off; **≈0.61** — 50 characters off;
- **≈0.37** — 100 characters off;
- **≈0.01** — 500 characters off.

The **final score is the mean** of per-sample scores over all samples in the split (reported on a 0–100 scale). The metric rewards getting *close*, not just exact.

Constraints

- **Environment:** one GPU (≈16 GB VRAM), no internet at grade time — the allowed model (below) is already provided. **Wall-clock budget: 10 minutes** for the whole run — this must cover any training / fine-tuning you do at grade time **plus** inference on the evaluation set.
- **Allowed pretrained model** — this is exhaustive; no other pretrained weights may be used. It is **pre-provided in the environment** (load it normally, e.g. `from_pretrained`; there is no internet at grade time):
 - **bge-base-en-v1.5** — a 110M-parameter text **encoder** (embedding model). It produces sentence/passage embeddings; it is not a generative language model. You may use it **as-is (frozen features)** or **fine-tune it on the train split** (full fine-tuning fits the 16 GB / 10-minute budget).
- Classical / statistical tools are unrestricted: you may build any feature-based model (e.g., scikit-learn classifiers or regressors) on top of embedding features you compute yourself. *Pretrained deep-learning weights* are only restricted to the list above.